

source =
*chatgpt.com)**
*localLaTeXeditor**
VSCode+
LaTeXWorkshop

Call Break Scoreboard Tool

Product Requirements Document (PRD)

Prepared For Development

June 3, 2026

Contents

1 Project Overview

1.1 Project Name

Call Break Scoreboard Tool

1.2 Objective

The objective of this project is to build a modern offline-friendly web application for tracking and monitoring Call Break game scores among 4 players.

The platform should provide:

- Fast score entry
- Real-time rankings
- Graphical score analytics
- Match history tracking
- Player management
- Offline usability

The application will be optimized for:

- Mobile devices
- Tablets
- Desktop browsers

2 Technology Stack

2.1 Frontend

- React
- TypeScript
- TailwindCSS
- Zustand (State Management)
- React Router
- Recharts / Chart.js
- PWA Support

2.2 Backend

- Node.js
- Express.js
- REST API

2.3 Database

- Supabase PostgreSQL

2.4 Deployment

- Frontend: Vercel
- Backend: Railway / Render
- Database: Supabase Cloud

3 Core Features

3.1 Player Management

- Add player
- Remove player
- Edit player details
- Save player profiles
- Player statistics

3.2 Match Management

- Create new match
- Select 4 players
- Arrange player order
- Start/Pause/End match
- Save match history

3.3 Scoreboard

- Round-wise score entry
- Running totals
- Live rankings
- Rank movement indicators
- Undo/Edit scores

3.4 Analytics

- Score progression graph
- Ranking progression graph
- Win percentage
- Average score
- Match history analytics

3.5 Offline Support

- PWA installation
- Local caching
- IndexedDB support
- Sync when internet returns

4 System Architecture

4.1 High-Level Architecture

Frontend (React) ↓
REST API (Node + Express) ↓
Supabase PostgreSQL Database

5 Frontend Architecture

5.1 Frontend Folder Structure

```
src/  
|  
|-- components/  
|   |-- scoreboard/  
|   |-- graphs/  
|   |-- players/  
|   |-- ui/  
|  
|-- pages/  
|   |-- Dashboard.tsx  
|   |-- MatchSetup.tsx  
|   |-- LiveMatch.tsx  
|   |-- MatchHistory.tsx  
|   |-- PlayerManagement.tsx  
|  
|-- services/  
|   |-- api.ts  
|   |-- matchService.ts  
|  
|-- store/  
|   |-- useMatchStore.ts  
|   |-- usePlayerStore.ts  
|  
|-- hooks/  
|  
|-- types/  
|  
|-- utils/  
|  
|-- routes/  
|  
|-- App.tsx
```

6 Frontend Modules

6.1 Dashboard

Displays:

- Recent matches
- Quick start
- Player rankings
- Statistics overview

6.2 Player Management

Features:

- Add/Edit/Delete players
- Player avatars
- Lifetime statistics

6.3 Match Setup

Features:

- Select 4 players
- Drag-and-drop ordering
- Configure rounds

6.4 Live Match Screen

Features:

- Score entry table
- Real-time ranking
- Round history
- Undo functionality
- Graphs

6.5 Analytics Screen

Features:

- Score graphs
- Rank progression
- Historical trends

7 Backend Architecture

7.1 Backend Folder Structure

```
server/  
|  
|-- controllers/  
|  
|-- routes/  
|  
|-- middleware/  
|  
|-- services/  
|  
|-- database/  
|  
|-- utils/  
|  
|-- app.js  
|  
|-- server.js
```

8 Backend Modules

8.1 Authentication Module (Optional)

- Guest access
- Optional login

8.2 Player Service

Responsibilities:

- Create player
- Update player
- Delete player
- Fetch player stats

8.3 Match Service

Responsibilities:

- Create match
- Add rounds
- Update scores
- Calculate rankings

8.4 Analytics Service

Responsibilities:

- Generate score graphs
- Rank movement tracking
- Statistical calculations

9 Database Architecture

9.1 Database Tables

9.1.1 users

Column	Type	Description
id	UUID	Primary key
name	VARCHAR	Player name
avatar	TEXT	Avatar URL
created_at	TIMESTAMP	Creation time

9.1.2 matches

Column	Type	Description
id	UUID	Primary key
status	VARCHAR	active/completed
winner_id	UUID	Winner reference
created_at	TIMESTAMP	Match creation time
ended_at	TIMESTAMP	Match end time

9.1.3 match_players

Column	Type	Description
id	UUID	Primary key
match_id	UUID	Match reference
user_id	UUID	Player reference
seat_order	INTEGER	Playing order

9.1.4 rounds

Column	Type	Description
id	UUID	Primary key
match_id	UUID	Match reference
round_number	INTEGER	Round number
created_at	TIMESTAMP	Round creation time

9.1.5 round_scores

Column	Type	Description
id	UUID	Primary key
round_id	UUID	Round reference
user_id	UUID	Player reference
score	INTEGER	Round score
bid	INTEGER	Player bid
actual_wins	INTEGER	Actual tricks won

10 API Design

10.1 Player APIs

- GET /players
- POST /players
- PUT /players/:id
- DELETE /players/:id

10.2 Match APIs

- POST /matches
- GET /matches/:id
- PUT /matches/:id
- DELETE /matches/:id

10.3 Round APIs

- POST /rounds
- PUT /rounds/:id
- DELETE /rounds/:id

11 Graph System

11.1 Score Progress Graph

Displays:

- X-axis = rounds
- Y-axis = cumulative score
- 4 player lines

11.2 Rank Progress Graph

Displays:

- Rank changes over time
- Position movement
- Overtake visualization

12 State Management

12.1 Global State

Managed using Zustand.

12.2 Stored States

- Current match
- Player data
- Rankings
- Match history
- UI preferences

13 Offline Strategy

13.1 Offline Features

- IndexedDB caching
- Service worker
- Local score persistence
- Background sync

13.2 Sync Strategy

- Local-first writes
- Queue unsynced actions
- Sync when online

14 Security

14.1 Security Features

- JWT authentication (optional)
- Rate limiting
- Input validation
- SQL injection protection

15 Performance Requirements

- Initial load < 2 seconds
- Score update latency $< 100\text{ms}$
- Offline support enabled
- Mobile responsive UI

16 Future Enhancements

- Multiplayer online sync
- Spectator mode
- Tournament system
- AI score predictions
- Voice score entry
- Export to PDF

17 Development Roadmap

17.1 Phase 1 - MVP

- Player management
- Match setup
- Scoreboard
- Rankings

17.2 Phase 2

- Graphs
- Analytics
- Match history

17.3 Phase 3

- Offline sync
- PWA support
- Realtime multiplayer

18 Conclusion

The Call Break Scoreboard Tool aims to deliver a fast, intuitive, and highly interactive experience for offline and online Call Break score management.

The architecture is designed to be:

- Scalable
- Mobile-first
- Offline-capable
- Maintainable